Transfer Learning Comparison Report

Feature Extraction vs. Fine-Tuning — MobileNetV2 on the Flowers Dataset

---

1. Scope of This Report

This report compares the two transfer-learning strategies implemented in feature_extraction.ipynb and fine_tuning.ipynb, using the actual results from the executed runs. Both notebooks used the same seed, image size, batch size, dropout rate, learning rates, epoch ceilings, and early-stopping settings, and loaded the identical dataset (3,670 images, 5 classes: daisy, dandelion, roses, sunflowers, tulips; verified ImageNet weights loaded successfully in both runs) — so the difference in outcome below reflects the strategy, not a mismatched setup.

2. Actual Results

Metric	Feature Extraction	Fine-Tuning
Test Accuracy	87.50%	89.49%
Test Loss	0.3685	0.3228
Epochs Trained (ceiling)	13 (of 30)	Phase 1: 13 (of 30) + Phase 2: 9 (of 20) = 22 total
Best Epoch (weights restored from)	Epoch 8	Phase 1: epoch 8 → Phase 2: epoch 17 (val_acc 90.31%, val_loss 0.2787)
Trainable Params (final)	6,405	1,867,845 (54 of 154 base layers unfrozen)
Macro Avg F1	0.88	0.89
Weighted Avg F1	0.88	0.89

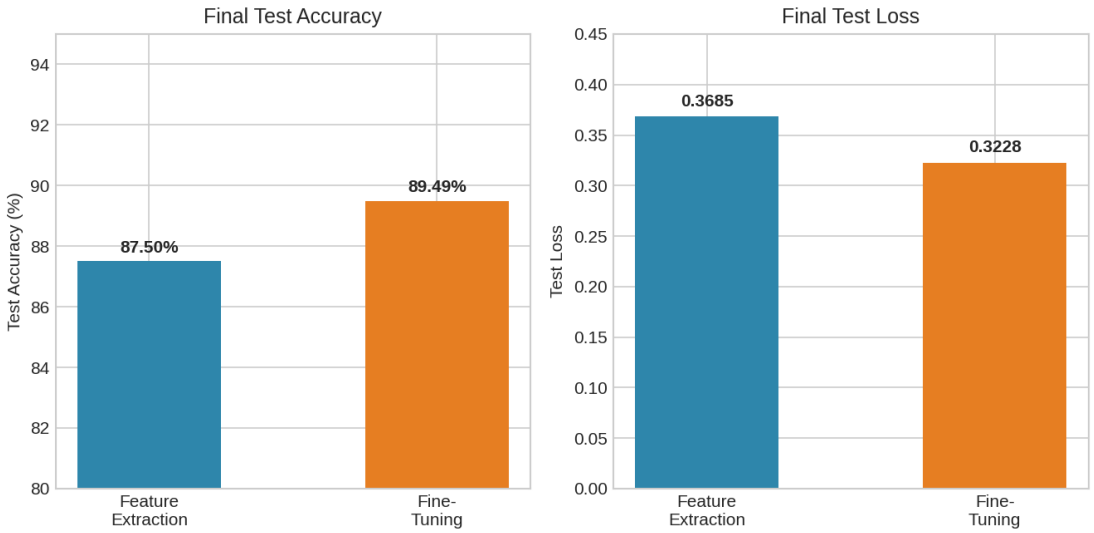


Figure 1. Final test accuracy and loss — fine-tuning improved accuracy by +1.99 points and reduced loss by 0.0457.

Fine-tuning improved test accuracy by +1.99 percentage points and reduced test loss by 0.0457 compared to feature extraction alone, using about 9 extra epochs at a 100x lower learning rate on 54 unfrozen base layers.

Per-Class F1-Score Comparison



Figure 2. Per-class F1-score — every class held steady or improved under fine-tuning; none regressed.

Class	Feature Extraction	Fine-Tuning	Change
daisy	0.90	0.91	+0.01
dandelion	0.93	0.94	+0.01
roses	0.83	0.85	+0.02
sunflowers	0.87	0.89	+0.02
tulips	0.86	0.86	0.00

Every class held steady or improved slightly under fine-tuning — no class got worse. Roses and sunflowers — the two lowest-scoring classes under feature extraction — saw the largest gains, consistent with fine-tuning helping most where the frozen ImageNet features were least sufficient on their own. Tulips didn't move at all, and remains (tied with roses) the weakest class in both runs — worth a closer look if you want to push further, since tulips vs. roses is a visually plausible confusion (similarly shaped, differently colored flowers, and color alone can be an unreliable cue under augmentation like RandomContrast).

Recall patterns shifted rather than just improving uniformly: dandelion's recall dropped slightly (0.97 → 0.92) while its precision rose (0.90 → 0.97), and sunflowers' recall jumped from 0.88 to 0.97 while its precision fell (0.86 → 0.82). This suggests fine-tuning redrew the decision boundary between some visually similar classes rather than simply making every prediction more confident across the board.

3. Training Curve Highlights

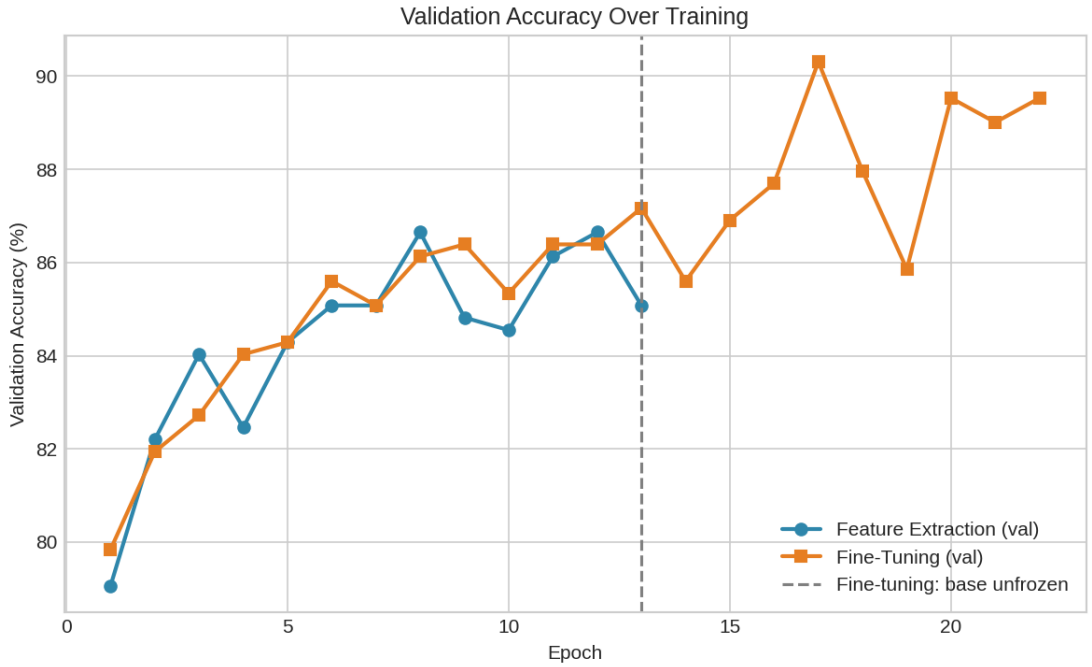


Figure 3. Validation accuracy per epoch. The dashed line marks where the base model was unfrozen in the fine-tuning run — note how closely the two curves track each other before that point.

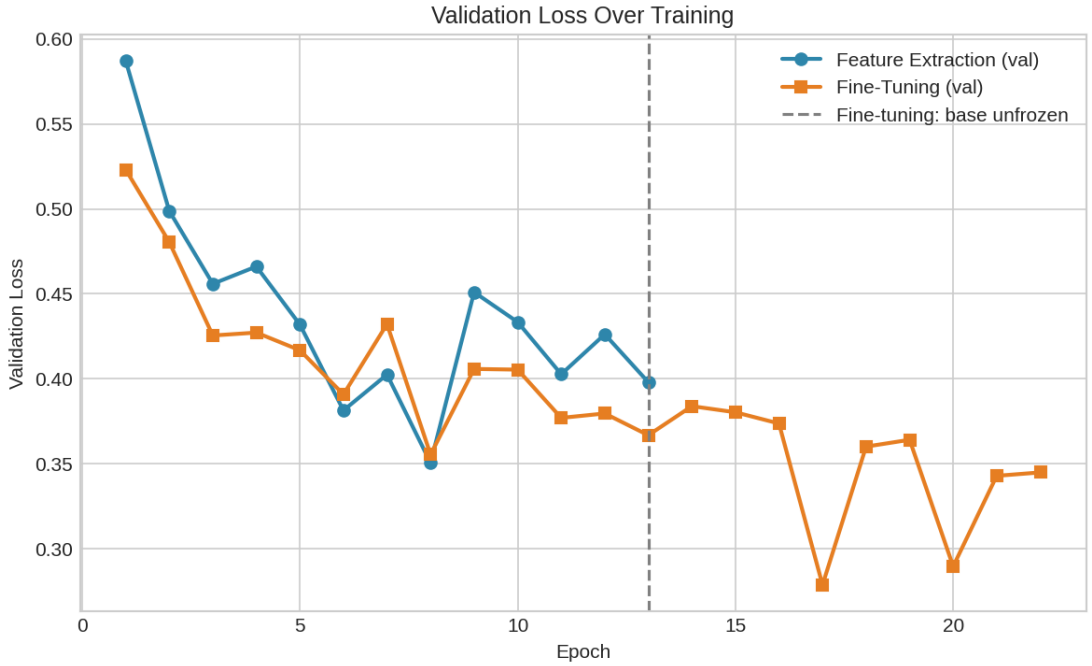


Figure 4. Validation loss per epoch. Fine-tuning's loss drops well below feature extraction's floor once the base is unfrozen (epoch 17), before early stopping ends the run.

- Feature extraction (single phase): train accuracy climbed from 63.9% → 90.4% over 13 epochs; val_accuracy peaked at 86.65% (epoch 8, restored) before val_loss crept back up — a contained, normal overfitting pattern that early stopping caught cleanly.

- Fine-tuning Phase 1 (warm-up): nearly identical trajectory to feature extraction — 63.8% → 88.9% train accuracy, val_accuracy peaking at 87.17% (epoch 8/13) — confirming the two notebooks' shared setup was genuinely equivalent up to this point.
- Fine-tuning Phase 2 (unfrozen, low LR): train accuracy dipped to 77.3% at the first unfreezing epoch (14) — the expected brief disruption from suddenly making 1.86M more parameters trainable — then climbed past Phase 1's ceiling, peaking at val_accuracy 90.31% / val_loss 0.2787 at epoch 17, before early stopping ended the run at epoch 22.

4. Methodology Side-by-Side

Aspect	Feature Extraction	Fine-Tuning
Base model state	Frozen for the entire run	Frozen during warm-up, then top 54 layers unfrozen
What actually trains	Classification head only (6,405 params)	Head (Phase 1) → head + top 54 base layers (Phase 2, 1,867,845 params)
Learning rate	1e-3 throughout	1e-3 (Phase 1) → 1e-5 (Phase 2, 100x smaller)
Training phases	Single phase	Two phases (warm-up, then fine-tune)
Early stopping	Patience 5 on val_loss, best weights restored	Applied independently in each phase
Relative compute cost	Lowest (13 epochs, head only)	Higher (22 epochs total; Phase 2 backprops through 54 extra conv layers)
Failure mode if misconfigured	Underfitting (head too simple)	Catastrophic forgetting (LR too high or too many layers unfrozen)

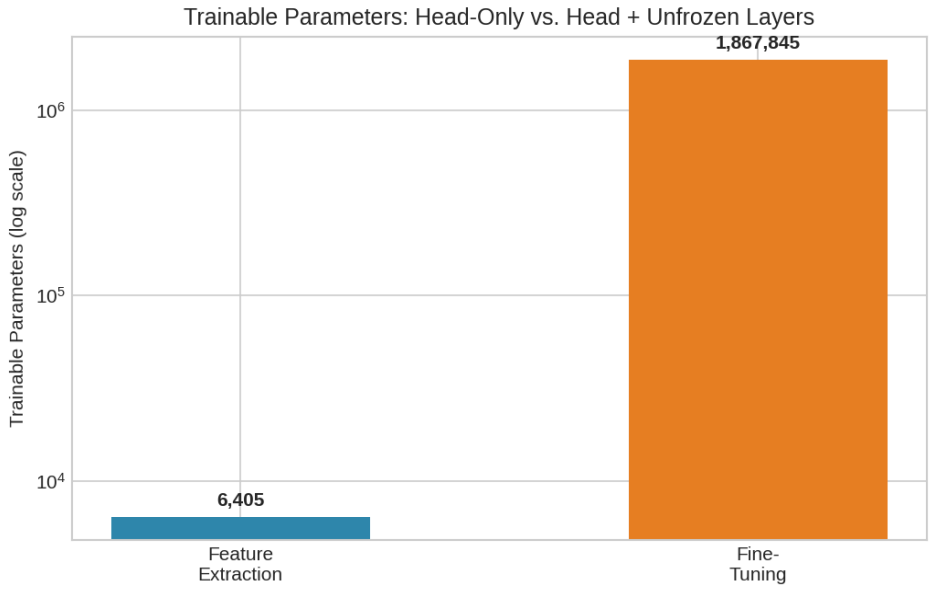


Figure 5. Trainable parameters (log scale) — fine-tuning trains roughly 290x more parameters than feature extraction.

5. Feature Extraction — Advantages & Drawbacks

Advantages

- Fast and cheap — only the head's gradients are computed; the whole run finished in 13 epochs of head-only training.

- Low overfitting risk — with only 6,405 trainable parameters against ~2,936 training images, the model has little capacity to memorize the training set.
- Stable and predictable — no risk of destroying the pretrained ImageNet weights, so results are reproducible and easy to reason about.
- Works with very little data — feature extraction is often the only viable option when a dataset is small.
- Good baseline before committing to fine-tuning — it's literally reused as the warm-up phase (Phase 1) inside the fine-tuning run, and its own accuracy (87.50%) already came within 2 points of the fine-tuned result.

Drawbacks

- Ceiling on accuracy — the frozen ImageNet features can't adapt to characteristics specific to the flowers dataset; in this run, that ceiling landed at 87.50%.
- No adaptation of mid/high-level features — layers that encode object-part and semantic features stay generic.
- Head-capacity bottleneck — a single GlobalAveragePooling2D → Dropout → Dense head has limited ability to compensate for features that aren't perfectly suited to the task, which shows up as roses and sunflowers being its weakest classes (F1 0.83 and 0.87).

6. Fine-Tuning — Advantages & Drawbacks

Advantages

- Higher achieved accuracy — 89.49% vs. 87.50%, with every class holding steady or improving; roses and sunflowers (the weakest classes under feature extraction) gained the most.
- Better handling of domain shift — unfreezing the top 54 layers let the network specialize its highest-level features to flowers specifically.
- Reuses feature extraction as a safety net — Phase 2 started from the already-warmed-up head from Phase 1, giving Phase 2 a stable starting point rather than starting from scratch.

Drawbacks

- Higher risk of catastrophic forgetting — this is why FT_LR (1e-5) was set 100x smaller than BASE_LR; the visible train-accuracy dip to 77.3% at epoch 14 shows how disruptive unfreezing can be even with a small learning rate.
- More prone to overfitting — 1,867,845 trainable parameters against the same ~2,936 training images is a much higher capacity-to-data ratio than feature extraction's.
- Slower and more expensive — 22 total epochs vs. 13, and each Phase 2 epoch is heavier since gradients flow through 54 extra convolutional layers.
- More hyperparameters to get right — FINE_TUNE_AT (54 layers here) and FT_LR both matter and interact; unfreezing too much or using too high an FT_LR degrades results, while too little of either makes Phase 2 barely different from Phase 1.
- Tulips didn't respond to fine-tuning at all (F1 stayed at 0.86) — a reminder that unfreezing more layers is not a universal fix for every class.

7. What Should Be Noted

- Phase 1 of fine-tuning tracked the feature-extraction run closely, as expected — both early-stopped at epoch 13/30 with best weights restored from epoch 8, and val accuracy differed by only ~0.5 points (86.65% vs. 86.13%). This confirms the two notebooks were genuinely held equal going into Phase 2.
- The accuracy gain from fine-tuning (+1.99 points) is real but modest, and came at real cost: 9 extra epochs, 1.86M additional trainable parameters, and a temporary accuracy dip when the base layers were first unfrozen.
- The test set is small — 352 images across 5 classes. A handful of examples flipping from correct to incorrect can move accuracy by roughly 0.3 points each, so the ~2-point fine-tuning advantage is on the edge of what this test set can reliably distinguish — a real signal, but not an enormous one.

- Tulips and roses remain the two weakest classes under both approaches, and fine-tuning barely moved tulips at all. If accuracy on those classes specifically matters, the fix likely isn't more fine-tuning — it's more training data or targeted augmentation for those two classes, since both approaches plateaued in the same place.
- Precision/recall shifted between classes rather than uniformly improving — fine-tuning redrew decision boundaries between visually similar classes rather than just sharpening every prediction.
- ImageNet weights loaded successfully in both runs (confirmed by the weights-file download log and the success message), so this is a genuine transfer-learning comparison, not one running on randomly-initialized weights.
- Model file size and inference cost are identical between the two (same final architecture) — the approaches differ in training cost and converged accuracy, not deployment cost.

8. Recommendation

- Fine-tuning is the better model here (89.49% vs. 87.50% test accuracy, no class regressed) — if only one final model is needed, ship the fine-tuned one.
- Feature extraction remains the better choice when training budget or time is tight — it reached 87.5% in 13 epochs of head-only training vs. fine-tuning's 22 total epochs plus a heavier per-epoch cost.
- To push past 89.5%, target tulips and roses specifically — more training images or augmentation tuned to reduce color-based confusion — rather than further fine-tuning epochs, since Phase 2 had already plateaued (early-stopped at epoch 22 after peaking at epoch 17).